

How to write a paper review

A strong review is clear, fair, constructive, and evidence-based, which will help *authors* improve their work and the *program committee (PC)* to make informed decisions. Below is what you should include and consider in each section.

Overall merit (score)

Options: Reject, weak reject, weak accept, accept, strong accept

The score represents your overall judgment of the paper's quality and contribution, whether the paper is worthy of publication, and should balance between strengths vs. weaknesses.

What justifies each score:

- **Reject:** major flaws or lack of novelty/contribution
- **Weak reject:** some merit, but significant issues
- **Weak accept:** good idea, but with noticeable limitations
- **Accept:** solid work, minor issues
- **Strong accept:** clear novelty, strong evaluation, high impact

Your score must be consistent with your written review. Avoid giving a high score but listing mostly critical weaknesses (and vice-versa).

Reviewer expertise

Options: No familiarity, some familiarity, knowledgeable, expert

This section depicts your level of confidence in evaluating this paper, as well as your familiarity with the topic, methodology, and related work. This helps the PC weigh your review appropriately.

Paper summary

The key idea is to summarize the paper (with your own words) to someone who is not familiar with it (for example, PC chairs or other reviewers). It should include the *problem* the paper addresses, the proposed *approach/solution*, the *key contributions*, and the *main results*. This will also show that you understood the paper correctly.

A good summary is neutral (no opinions yet), concise (typically 1 to 2 paragraphs), and accurate. Avoid copying sentences from the paper and adding critiques (save that for later).

Strengths

(Almost) Every paper has good things to point out. This section usually includes 2-3 sentences that highlight the strong points of the paper.

What to highlight:

- Novelty and originality
- Technical quality
- Sound methodology
- Strong experimental evaluation
- Clarity and organization

Good practice: be specific! Avoid stating that *“The paper is good.”* ... instead, discuss specific points of the paper *“The experimental evaluation is comprehensive, covering multiple realistic workloads and compares against strong baselines.”*

Weaknesses

All papers have limitations (it is very unlikely that a paper addresses all concerns of a field). This section usually includes 2-3 sentences that highlight the paper's weaknesses.

What to highlight:

- Missing or unclear contributions
- Weak or incomplete evaluation
- Incorrect assumptions
- Lack of comparison with relevant work (both related work and evaluation baselines)
- Poor clarity or organization
- Overstated claims

Good practice: be constructive, not dismissive! Focus on issues that matter. Avoid stating that *“The paper is bad.”* ... instead, discuss specific points of the paper *“The experimental evaluation lacks comparison with prior work X, making it difficult to assess novelty.”*

Detailed comments

This is the most important section. Here you should make a deep analysis of the contributions, proposed techniques, methodology, experimental setup, result interpretation, claims vs. evidence, etc. You *must* provide concrete suggestions for improvement, and you can include questions for the authors, and clarifications.

Structure (recommended)

Instead of creating bullets or independent comments as you go through the paper, categorize into:

- Major issues (blocking concerns)

- Minor issues (clarifications, presentation)
- Suggestions for improvement

Good practice: be precise and actionable. Provide references to specific parts of the paper (e.g., “*Section 4.2 ...*”), distinguish critical flaws vs. nice-to-have improvements.

Comments for PC members (optional)

This section is optional, and it does not replace the discussion among reviewers during the “discussion period”. Here you can include concerns you prefer not to share with the authors, such as suspicion of lack of novelty or overlap with known work, ethical concerns (use of AI without explicit acknowledgment, plagiarism, fake results, etc), or confidence in your evaluation.

Important: Do not introduce entirely new technical criticisms here that are not in the main review.

Matchmaker Paxos: A Reconfigurable Consensus Protocol

Paper: <https://openreview.net/pdf?id=bXe1agiq9LN>

Original reviews: <https://openreview.net/forum?id=bXe1agiq9LN>

Review A

Score: Weak accept

Expertise: Expert

Summary

Donut Paxos is a variant of MultiPaxos that uses separate nodes for configuration management. It addresses the problem of missing reconfiguration protocols in academic consensus protocols. Performance is on par with horizontal MultiPaxos and Raft.

Strength

- This paper focuses on the reconfiguration problem. It may give more insights on reconfiguration to readers.

Weakness

- Weak motivation.
- The paper's solution is missing an important component.

Detailed comments

I got many questions after reading through this paper. The primary concern is that this paper is missing a good motivation. I agree that reconfiguration is an important piece for a consensus protocol to be practical. But in the industry, consensus protocols are mostly used for configuration service, which manages replicas of primary backup replication. Even for the case when Paxos-like consensus protocol is used for main applications, replacing a failed server can be handled by the network layer (e.g., updating DNS to redirect requests to a new server). Or simple horizontal Paxos-like approach may work fine. To motivate Donut Paxos well, there should be some explanation on why it is important to provide reconfiguration service directly within consensus protocols.

Another problem is Donut Paxos doesn't provide a complete solution to the reconfiguration problem in state machine replication. Donut Paxos only covers how to safely reconfigure Paxos acceptors and ensure that servers can continue to reach an agreement. However, the difficulty of reconfiguration comes with recovering the state machine on a new replica. When Google first implemented MultiPaxos, they struggled a lot with the subtleties with recovering state machines. (Lamport's paper doesn't cover this topic.) On the other hand, Raft could be adopted widely in the industry since it fully specified how to handle the recovery of state machines, including

recovery from a snapshot (they are discussed in Diego's PhD thesis). Without disclosing how to recover the state machine safely, Donut Paxos has the same problem as horizontal MultiPaxos, which will prevent the industry's adoption again.

Lastly, the current introduction implies that the existing reconfiguration protocols cause performance degradation and that Donut Paxos has superior performance to them. However, in the evaluation section, Multi-Paxos's performance is the same as Donut Paxos's. It was a surprise when the evaluation section admits that the performance is not the contribution of Donut Paxos. I think this should be made clear earlier.

My suggestions are

- Clarify contribution: I think Donut Paxos can be valuable if it can be applied to any Paxos-based protocols. To show that, I think it should show another example, maybe applying Donut Paxos to EPaxos. It's a bit disappointing to see just the Multi-Paxos example as it already has horizontal reconfiguration.
- Extend protocol to the full solution. If you believe that missing reconfiguration was the reason why the industry didn't adopt improved Paxos variants, you should provide the full solution.
- I would conduct a survey on how industry people use consensus protocols and what problems they are suffering from. Then, Donut Paxos can be motivated more strongly.

Miscellaneous comments

- Page 1. "inefficient reconfiguration protocols [15, 22]" ==> can you evaluate their performance and show Donut Paxos's performance advantage?
- Page 1. "4% effect on the median of throughput and latency measurements" => what does it mean? Median of the whole experiment with a failure? If that's the case, medians cannot capture the reconfiguration behavior. If this is referring to the experiment in Section 7.1, the paper said reconfiguration completes within a millisecond, and the median was captured within one second window. I think that the "4%" number will change if a different time window was chosen.
- Page 3. "MATCHA" and "MATCHB" are a bit confusing.
- Page 4. "The proposer then ends the Matchmaking phase" ==> Is there a termination guarantee?
- Page 4. "to every acceptor in every configuration in H_i " ==> Isn't it just 1 per round?
- Page 4. In Algorithm 3, "an arbitrary configuration" ==> is it truly arbitrary?
- Page 4. In Algorithm 3, "Phase 1 quorum from every configuration in H_i " ==> what is the size of quorum?

Review B

Score: Weak reject

Expertise: Expert

Summary

The paper presents Donut Paxos and Donut MultiPaxos which are a reconfigurable consensus protocol and reconfigurable replicated state machine, respectively. The primary value of the proposed protocols is that they allow quick reconfiguration of the active set of acceptors while having no performance degradation. The proposed protocol performs reconfiguration off the critical path of standard command processing by employing a set of matchmaker processes. Matchmakers implement a log that stores the current and previous configurations (i.e., history of all configurations). Also, Donut Paxos/MultiPaxos uses vertical reconfiguration instead of the classical horizontal configuration that is used in Paxos. Vertical reconfiguration enables the system to perform reconfiguration quickly without the need to wait for the execution of some user commands. The evaluation results show that Donut Paxos/MultiPaxos performs reconfiguration quickly (orders of milliseconds) and does not have any overhead on the systems performance.

Strength

N/A

Weakness

N/A

Detailed comments

The paper addresses an interesting problem that is usually poorly discussed in papers proposing consensus protocols. I like the way in which the solution was presented starting from building a reconfigurable consensus protocol (Donut Paxos) and then extending it to build a reconfigurable replicated state machine (Donut MultiPaxos). Also, I commend the authors for having a detailed proof of the safety of the protocol as well as for discussing how other consensus protocols can be extended to implement vertical reconfiguration.

However, I mark this paper as weak reject for several reasons. First, I have some concerns about the novelty of the solution; you can think of the matchmakers as a separate state machine that is used to manage the configurations. That is, rather than having a single state machines for both user commands and configurations, Donut MultiPaxos decouples them and has one for each. Moreover, there are $2f+1$ matchmakers and a proposer waits for $f+1$ (typical majority) matchmaker responses to write current set of acceptors and get all previous configurations.

Second, this decoupling between configuration and command processing accelerates configuration changes. However, it increases the probability of unavailability as the availability of the commands' state machine is tightly coupled to the availability of the matchmakers' state machine. That is, if the matchmakers' state machine is not available, a new leader/proposer cannot execute any command even if all acceptors are up and running. The paper does not discuss this issue or evaluate it.

Third, the paper addresses the reconfiguration of the acceptors only and do not discuss the reconfiguration of replicas. That is, the only place in which the paper discusses that is in the following excerpt: “Replicas can also be safely added or removed at any time so long as we ensure that commands replicated on $f + 1$ replicas remain replicated on $f + 1$ replicas.” However, I believe that the issue of making sure that commands are always replicated on majority of replicas is very tricky and this is what guarantees the safety of a replicated state machine.

Fourth, Donut Paxos was not rigorously evaluated. For instance only 8 clients were used which is very small number of clients and not enough to stress the system. Also, the experiments do not highlight any benefits for Donut MultiPaxos over MultiPaxos in terms of throughput and latency. For better evaluation and better presentation of results, I suggest having a throughput-latency graph that compares Donut MultiPaxos and MultiPaxos for different number of clients and different alpha values (i.e., concurrent commands). The following are other minor notes I have about the experiments:

- The results shown in Figure 9 and 10 are confusing; why Donut MultiPaxos has higher throughput than MultiPaxos in the first 10 seconds (no reconfigurations occur)?
- Figure 14 and its caption does not match the text used to describe it in section 7.2.
- Having Donut MultiPaxos and MultiPaxos results in the same Figure will make it easier to read and compare results.

Finally, a minor point, I found that the text is not accurate in some places:

- “Many state machine replication protocols do not have logs and cannot perform horizontal reconfiguration [2, 8, 27, 30, 33]” (Introduction). The citations include EPaxos and Raft and both have logs.
- Section 2.2 states that Paxos requires $f+1$ proposers, and I think it does not, and it works fine with any number of proposers.
- In Figure 4, should the number of replicas be $2f+1$?
- “Raft cannot safely perform horizontal reconfigurations” (Section 6). I believe that Raft reconfiguration is based on the replicated log.
- “protocol. However, while none of the protocols have logs” (Section 6). This is not accurate.

Review C

Score: Weak accept

Expertise: Knowledgeable

Summary

N/A

Strength

N/A

Weakness

N/A

Detailed comments

Donut Paxos is a very good paper that proposes a solution to an actual problem (a reconfiguration of the consensus based systems) which covers both the most widespread log-based systems (multi-paxos, raft) and the promising alternative systems such as EPaxos.

It contains a lot of good practical optimizations and it will definitely benefit the community. However I recommend doing another iteration before accepting the paper because some of its parts look misleading or contradict each other.

The main idea, a reconfiguration protocol, is excellent. It takes from Vertical Paxos and then goes forward with replacing a master with a specialized linearizable replicated service which doesn't rely on consensus and has trivial reconfiguration protocol. Also Donut Paxos improves the GC procedure. I find this approach much easier to understand and to reason about compared to horizontal variants such as joint consensus. One of the interesting implications of this approach is that it helps to get rid of the control messages which often looks like gaps to an application which uses consensus as a foundation for the replicated log service.

Another great idea is the communication protocol between the proposers and the matchmaker services. It combines linearizable querying and updating into a single command, executes it with a single round trip and the protocol doesn't suffer from contention under concurrency (like dueling proposers do with Fast Paxos).

Last things I want to highlight is the formalization of the "phase 1 bypass" technique via the triple ballot numbers and the formalization of the pipelining via counting inflight requests (k). Those ideas (bypass, pipelining) have been around since "Paxos Made Simple" but often they're left underspecified and finding info on how to do it becomes a rite of passage for an engineer implementing a consensus based system.

What I find concerning is that the authors claim that reconfiguration takes only one round of communication. While being technically correct it feels misleading. Engineers measure duration of the reconfiguration from the moment it's started to the moment it's safe to shut down a node. Since the usual deployment of the consensus based systems in the industry consist of $2f+1$ nodes with colocated roles (Etcd, Zookeeper, CockroachDB, Redpanda etc.) having the replicas running on the same nodes as acceptors requires to copy its state before turning a node off (to satisfy GC scenario #3). From this position the following statements "reconfiguring to a new set of machines takes one round trip of communication" and "all of our results hold in a co-located deployment as well" contradict each other.

Another issue is the underspecified replica reconfiguration protocol and the unclear responsibility of the replica nodes. The sentence "replicas can also be safely added or removed at any time so long as we ensure that commands replicated on $f+1$ replicas remain replicated on $f+1$ replicas" isn't aligned with "state machine replicas execute the commands in log order".

After reading them it's unclear whether replicas store commands after execution and how long they should do it.

Next issue is related to determinism of Paxos rounds: "every round is orchestrated by a single predetermined proposer". This sentence creates an impression that given a current round (a ballot number) it's possible to predict the next one but it isn't true for example after (4,"p1") there may be (4,"p2"), (5,"p1") or any other ballot number which is lexicographically greater than the current round.

A related thing is using $i+1$ notation for the next round. The next round (ballot number) isn't defined until it happens so defining it as a function from the previous round isn't right. Also operation $+1$ isn't defined on tuples ("let the set of rounds be the set of lexicographically ordered integer pairs (r, id) where r is an integer and id is a unique proposer id"). It might be better to use "next round" instead of $i+1$ or to use subscripts r_i and r_{i+1} instead of i and $i+1$.

The last disturbing part is the evaluation section: the experiment is subject to the coordinated omission problem, it's unclear why the median and IQR metrics were chosen and the conclusion is self contradicting.

Clients wait to receive response before issuing the next command. This setup is subjected to the coordinated omission problem, a phenomenon when the measuring system coordinates with the measured system resulting in hiding the long periods of unavailability behind a single outlier which may be ignored even if we look at p99.

The evaluation part lacks a discussion why median and IQR are the important metrics. Even when the experiment is set up to avoid coordinated omission the reconfiguration still may significantly delay the requests without affecting the chosen metrics. For example when reconfiguration happens every second, lasts 250ms (a quarter of a second) and blocks the overlapping requests it doesn't affect IQR (a function from p75) but delays the requests by 850% (from 0.29ms to 250ms).

The following statements from the evaluation section contradict each other:

- "we include the comparison to MultiPaxos for the sake of having some baseline against which we can compare Donut MultiPaxos, but the comparison is shallow"
- "Donut MultiPaxos does provide performance benefits over MultiPaxos' and Raft's reconfiguration protocols"

I recommend to redesign the experiment to avoid the coordinated omission problem by maintaining the constant rate of the requests, come up with meaningful metrics (e.g. to compare statistics like p99 and max latency of the requests overlapping with the reconfiguration and not overlapping) and back up performance benefits of Donut Paxos with numbers.

Algorithmic Heap Layout Manipulation in the Linux Kernel

Paper: <https://openreview.net/pdf?id=UqszJh5h6v>

Original reviews: <https://openreview.net/forum?id=UqszJh5h6v>

Review A

Score: Weak accept

Expertise: Knowledgeable

Summary

The paper focuses on Linux kernel heap layout manipulation and exploitation, and it brings forward a number of contributions: 1) the development of a framework to evaluate heap layout manipulation algorithms for the Linux kernel, 2) a performance benchmark to evaluate these algorithms, and 3) the proposal and evaluation of two algorithms, random search and KEvoHeap, based on genetic algorithms. The majority of these techniques were already published by Heelan et al., but they were focusing on userspace heap: the net contribution of this paper is to port these techniques to the Linux kernel, which is not trivial.

Strength

- interesting work in an interesting research area
- paper is overall easy to read, and it contains a lot of background information
- the paper makes it very clear what it took from previous work and what are the "net" contributions
- it seems that (at least part of?) the work is / will be open sourced

Weakness

- the proposed approach, benchmark, and algorithms are all taken from existing works (but there are some tweaks that were necessary)
- the threat model is unclear (and even if one can guess, it should be spelled out)
- no concrete evaluation in realistic settings

Detailed comments

Thanks the authors for this submission! I enjoyed reading this paper. I'm not a super expert on kernel heap exploitation, but I believe I was able to read and understand the paper without problems. I appreciate the authors took the time to discuss a number of background information and technical details before diving into the technical contribution of the paper.

The contributions are heavily based by existing work from Heelan et al., who were focusing on userspace heap attacks. The main contribution of this paper is to port the techniques/contributions by Heelan (framework + approach) from user-space heap layout

manipulation/exploitation to the Linux kernel's. Even though the vast majority of the proposed techniques appear exactly the same, I really appreciate that the authors do not hide it and state the similarity (or the exact match) upfront; and, the authors provide a good discussion 1) explaining how porting these techniques from user space to kernel space was not trivial, and 2) explaining the rationale behind the difference. This makes the reader's job much easier. Thanks for that. Overall, I believe the difference are indeed not trivial, and the discussion and implemented differences are valuable. This work is also going to be open sourced (at least in part? see below), so it should be possible to build on top of it.

With that being said, I do have a number of concerns for this paper.

The first aspect is that the threat model is not clear, and even if one could guess it, I believe it should be properly spelled out. It also doesn't help that a reader familiar with kernel exploitation may assume a specific threat model... which then appears to be the wrong one. More in detail, when reading a paper about kernel exploitation, my assumption is that the threat model is "the attacker has arbitrary user-space code execution, and she wants to exploit a bug in the kernel to become root or achieve kernel-level code execution". Thus, I would have assumed that the attacker does not have access to root-level information or capabilities. But, when reading the paper, it becomes clear that the proposed system/mechanism does rely on "privileged information or mechanisms", such as accessing `/proc/slabinfo`, `kprobes`, or even having access to external QEMU resets. These mechanism are, of course, not accessible to an attacker who wants to perform local privilege escalation... and from this, I guess the authors are assuming a different threat model: not an attacker who wants to actually mount a LPE attack, but a vulnerability researcher who wants to show the exploitability of an heap-related bug that she found. Is my understanding correct? Regardless, I believe it's very important to be up front with the assumed threat model and which tools are available to the attacker/vuln researcher.

Another problem of the paper is that it is often not clear what are the implications of some "shortcuts" used to make the approach/discussion tractable. E.g., focusing on using a custom cache: this is mentioned in page 8, but the potential real-world applicability repercussions of this choice are discussed (in part) at the end of the paper, ~8/9/10 pages later. There are a number of similar decisions that simplify the implementation and evaluation of the proposed approach, but I believe it should be made crystal clear which decisions are "OK" and repercussion-free in terms of real-world applicability, and which are actually constituted a simplification that does make the discussion less realistic.

My biggest concern is about the applicability of this approach in the real-world, for what concerns a vuln researcher who wants to prove her real-world bug is exploitable, and for what concern an attacker who would like to use this approach to be assisted for the exploitation. While there are many "stability guarantees" when one deals with userspace heaps, the situation is much messier (to the point of unexploitability without additional bugs?) when dealing with kernel-level heaps. While the paper presents an evaluation on the synthetic dataset (+ many assumptions!), the paper only has a discussion-level evaluation for what concerns using this system in a real-world scenario... and still, the real-world scenario considered is not the one of

an attacker, but one in which the vuln researcher already has privileged access and "just" wants to show exploitability. What is the hope that such an approach could be used to assist in real-world exploitation? Under which assumptions (e.g., 'the heap is already defragmented', 'the initial state of the heap is know') would this tool be useful in real-world scenarios?

One comment on open sourcing: the paper mentions in a couple of points in the paper that "the code is on GitHub [18]". It is not clear whether everything about this paper is / will be on github, or just the two parts that were specifically in the paper? Would the authors mind clarifying whether the github repo contains all the artifacts? If that's the case, I suggest the authors to add a pointer about this aspect at the end of the introduction so that it's clear that everything will be available. It is a very important (positive) component, and I believe it's important to point this out!

Nitpick: the English is too informal at points. See usage of "e.g." (which, btw, should be "e.g.,") in the middle of sentences, or terms such as "huge", "skyrocketing", etc.

Some related works that the authors may want to consider adding, at least for context (nothing of this appears to be a critical omission, but these could be good to contextualize existing works in the area):

- HeapHopper: Bringing Bounded Model Checking to Heap Implementation Security, USENIX Security 2018
- Playing for K(H)eaps: Understanding and Improving Linux Kernel Exploit Reliability, USENIX Security 2022

Review B

Score: Weak reject

Expertise: Knowledgeable

Summary

The authors propose a new framework (Kernel-SIEVE) for evaluating heap layout manipulation algorithms in kernel space. They implement and compare two algorithms: a naive pseudo-random search, and a more advanced genetic algorithm, KEvoHeap. The genetic algorithm the authors present performs well even in realistic conditions where multiple concurrent threads are making use of the kernel heap.

Strength

N/A

Weakness

N/A

Detailed comments

Pros

- First attempt at evaluating kernel heap manipulation techniques: The framework introduced by the authors is extensible and looks like it can be used to evaluate more techniques.
- In-depth explanation of the details: The authors extensively describe both the algorithms, without excluding implementation details. This immensely helps reproducibility. Furthermore, the authors provide detailed explanations and examples to justify the results of their evaluation. I also appreciated the effort the authors put into writing the background sections: they introduce the problem space well and guide the reader into the necessary concepts for understanding the rest of the paper.
- Insights in the porting of SIEVE to kernel space: The real strength of this paper are the subtle fixes and clever workarounds that were necessary to port a heap manipulation framework to kernel space. In particular, I found the creation of a custom kernel cache for SLUBs a smart hack that I did not know was possible.

Cons

- Wrong assumption on CONFIG_SLAB_FREELIST_RANDOM: In page 4, the authors state that the free list randomisation kernel feature is out of scope for this study, due to it being disabled in the Linux default configuration. Unfortunately, while it is true that is disabled in the Linux default configuration, individual distributions often tweak and ship custom kernel configurations. I manually tested only Ubuntu and Arch Linux, but in both I found the CONFIG_SLAB_FREELIST_RANDOM to be enabled by default. I suspect that the vast majority of Linux distributions have this security feature enabled. This is a major issue, as it would seriously undermine the applicability of the authors' approach in real world scenarios. The authors state that there might be workarounds for this issue; they should do a deeper study on what they could be and if they will affect the KernelSIEVE's evaluating abilities.
- Limited novelty: Most of the work represents an extension of the work by Heelan and his SIEVE framework for userspace heap manipulation. This limits the scientific novelty to the insights required to port Heelan's approach to kernel space. From my understanding, the only significant modifications to Heelan's approach are those required to make it work in the kernel (other than minor changes in the evolution selection of KEvoHeap). The introduction and evaluation of a new algorithm other than the two Heelan proposed would definitely improve this paper. Nevertheless, the authors' extensively clarify which idea was taken from prior work and which idea instead was introduced in this paper, in a nice display of scientific correctness.
- Limited evaluation: The current evaluation is not enough to convince a reader of the real-world applicability of their work. A table such as Table 2 is necessary for the QEMU evaluation, as Figure 18 alone is not enough (especially considering that some values were estimated instead of measured). If the QEMU reset is too big of a performance bottleneck, the authors should look into alternative solutions (an example might be state-of-the-art snapshot fuzzers such as Nyx, that achieve thousands of executions per second)

Writing issues

While the paper was generally well-written, I spotted the following minor issues:

Looks like there is a misplaced e.g. in the first page (near “Zerodium”)

“Which’s” in the abstract

Page 8: no capitalisation after colon

Too much focus on technical details, too long paper:

Many parts of this paper (especially section 5 and 6) go into too much technical details of the implementation; I feel like they should best be placed in an appendix or in a README of the source code. So much information distracts the reader from the important design choices. I feel like the same concepts could be expressed in around 10-12 pages (instead of the current 18). Another example is Figure 6: the text describing the system is very clear and the figure does not add much, while still consuming almost half a page.

Conclusion

While the paper is an extension of a prior technique, and the proposed approach has limited applicability in the real world, I still think that some of the insights presented here are interesting and should be shared with the rest of the scientific community. For this reason, I will push for a weak accept of this paper. It is vital though that the authors address the randomisation of freelists, and do a more thorough study of applicability in the real world.

Review C

Score: Weak reject

Expertise: Knowledgeable

Summary

The authors analyse the problem of heap layout manipulation for the linux kernel and attempt to find solutions to automate this step. The paper starts with an analysis of the problem in general and of the specific challenges faced when solving this problem for the linux kernel. The authors go on to review the design of kernel memory management and then present their framework as an extension of the work from Heelan et al. The paper presents two algorithms (also inspired from prior work) and benchmarks their effectiveness against the allocator in the linux kernel. The authors present some concluding remarks and discussions.

Strength

- the paper is well written
- the problem is relevant and worthy of attention and research
- the paper is very informative: it contains a useful introduction to the problem and to the memory management internals of the linux kernel

Weakness

In the detailed comments

Detailed comments

Thanks for your submission. I'm happy to see activity in this space: the problem you are investigating is interesting, relevant and timely. The way you approach the problem is also the right one in my opinion, and I would like to see this paper published.

In order to get there, in my opinion, you should focus on the following areas:

- Tighter coupling to the linux kernel: I would recommend to take one extra step and reason in terms of system calls rather than in terms of invocations of `kmalloc` or analogous dynamic memory management API in the kernel. Basically, with Heelan's work in mind [33], I think the value of the paper would be immensely higher if you worked towards a SHRIKE (section 3.2 of [33]) for the kernel. My request could be motivated and justified by the following observations: 1) without this aspect, one is left to wonder whether you have "only" built (*) a new allocator to use SIEVE with (on top of `avrlibc`, `dlmalloc` and `tcmalloc`). 2) one is also left to wonder whether you could have achieved the same benchmarking setup by extracting the relevant dynamic memory management code of the kernel, built it in userspace and just used the original SIEVE framework (<https://github.com/SeanHeelan/HeapLayout>). I would recommend to invest some work into what you propose in section 9.1: analyse which system calls trigger which allocations and drive KSIEVE through a set of system calls rather than just invocations of `kmalloc`.
- complete the kernel memory management picture: there are some tradeoffs when using `kmalloc` directly vs. using `kmem_cache_alloc` and similar calls. Also, there are generic slabs vs dedicated caches, `vmalloc`, etc. Given the excellent intro you have provided to kernel memory management, I would encourage you to complete the picture which is surely going to be an interesting reference for readers interested in learning these aspects.
- exploit more deeply the fact that you're using the kernel allocator. As you point out at the end of section 4.2, the SLUB allocator has several peculiarities that make it unique and very different from `dlmalloc`, `tcmalloc` etc. I would have liked to see these considerations feature more prominently in your analysis and in the results/discussions. For example: you mention the possibility of metadata corruption via overflowing into a free block. This very interesting consideration doesn't seem to ever be used/discussed. Same for the bit about page frame borders or adjacent objects of the same size but different type. (incidentally, isn't the same true for `dlmalloc`?)
- clearer threat model: it's probably worth clarifying what the threat model is (early in the paper), the gap between the results that you will obtain/have obtained and the knowledge available to an adversary under your threat model. For example, as you only later point out, `/proc/slabinfo` is not world-readable and so it's probably worth clarifying earlier on that such information is not available to the adversary. It would also be interesting to clarify what kind of infoleak would be necessary to at least partially bridge this gap in a more practical scenario. For example, how what chances would an attacker have to prototype on a test system a special "reset" sequence followed by a sequence of allocations that lead to the desired heap configuration, and then reproduce this on a

target system? I was hoping to read some of that in section 9.2, but it focused instead on stabilising results in a more synthetic scenario.

- clearer "delta" with respect to Heelan's thesis. There's nothing wrong to build on top of existing research, but I think it would be very beneficial to clearly spell out what it is that your paper brings on top of the existing work.
- deeper explanation of the used frameworks. For example, section 6 seems to require intimate knowledge of Heelan's work. For example, `AppendFstSequence` is mentioned there without any kind of introduction.

In what follows I have collected some minor points:

- you mention heap feng shui, heap grooming or heap layout manipulation in section 3. Given that there are no space limits, could you say a few words about them and clarify how they are different?
- in section 4.1, you say that "when this object is allocated again, there is no need to initialise it again". What exactly do you mean? Surely some initialisation is still necessary - could you please clarify?
- in section 4.2 you refer to struct page assuming that the reader knows what it is/does. You might want to be a bit more clear in that regard.
- in section 4.2 you mention "overflowing between page frame borders [42]"; it would be very interesting to spend more time on presenting what this is (and as I mentioned above, in further evaluating how this aspect influences the problem of heap layout manipulation).
- how exactly do you introduce noise in the synthetic benchmarks (re section 7.1)?
- let me preface this by saying that the paper is generally well written. While this is only a matter of personal taste, I would encourage you to use a more formal writing style. In particular, I would
 - always use the present tense
 - avoid open questions in the paper
 - avoid exclamations or colloquial expressions (easier said than done)
- a few typos:
 - "Which's"
 - "can not" (multiple times)
 - "exemplary"

(*) and in saying only I'm not discounting all the good work that went into building the system, I'm merely attempting to measure the delta in terms of contributions and lessons learned that the paper is offering to the community